

Week 9

C237:

Software Application Development

LESSON 17:

Database Integration with Express II – Update & Delete



Recap

Route to retrieve ALL products

```
// Define routes
app.get('/', (req, res) => {
  const sql = 'SELECT * FROM products';
  // Fetch data from MySQL
  connection.query( sql , (error, results) => {
    if (error) {
      console.error('Database query error:', error.message);
      return res.send('Error Retrieving products');
    }
    // Render HTML page with data
    res.render('index', { products: results });
  });
});
```

Route to retrieve ONE product by id

```
app.get('/product/:id', (req, res) => {  
  // Extract the product ID from the request parameters  
  const productId = req.params.id;  
  const sql = 'SELECT * FROM products WHERE productId = ?';  
  // Fetch data from MySQL based on the product ID  
  connection.query( sql , [productId], (error, results) => {  
    if (error) {  
      console.error('Database query error:', error.message);  
      return res.send('Error Retrieving product by ID');  
    }  
    // Check if any product with the given ID was found  
    if (results.length > 0) {  
      // Render HTML page with the product data  
      res.render('product', { product: results[0] });  
    } else {  
      // If no product with the given ID was found  
      res.send('Product not found');  
    }  
  });  
});
```

Route to add product

```
app.get('/addProduct', (req, res) => {
  res.render('addProduct');
});

app.post('/addProduct', (req, res) => {
  // Extract product data from the request body
  const { name, quantity, price, image } = req.body;
  const sql = 'INSERT INTO products (productName, quantity, price, image) VALUES (?, ?, ?, ?)';
  // Insert the new product into the database
  connection.query( sql , [name, quantity, price, image], (error, results) => {
    if (error) {
      // Handle any error that occurs during the database operation
      console.error("Error adding product:", error);
      res.send('Error adding product');
    } else {
      // Send a success response
      res.redirect('/');
    }
  });
});
```



Update product details in database

Steps to update data in database

- 1. Display Current Data**
 - Retrieve and display existing data in a form.
- 2. User edits form with new data**
 - User updates the information
- 3. User Submits Changes**
 - User clicks “Update” to **send changes to the server.**
- 4. Update database**
 - Server updates the record with the new data.
- 5. Show Confirmation**
 - Notify user that the update was successful.

Update index.ejs page

- Open the **supermarketApp** created last week in Visual studio code.
- Edit the **index.ejs** page to display the “edit” and “delete” links.
- Don’t forget to add the headers <th> too.

```
<% for(let i=0; i < products.length; i++) { %>
  <tr>
    <td><a href="/product/<%= products[i].productId %>" ><%= products[i].productName %></a></td>
    <td><img src = "<%= products[i].image %>" width="20%"></td>
    <td><%= products[i].quantity %></td>
    <td>$<%= products[i].price.toFixed(2) %></td>
    <!-- Edit Link -->
    <td><a href="/editProduct/<%= products[i].productId %>">Edit</a></td>
    <!-- Delete Link -->
    <td><a href="/deleteProduct/<%= products[i].productId %>"
      onclick="return confirm('Are you sure you want to delete this product?')">Delete</a></td>
  </tr>
<% } %>
```

Display Current Data in form

- Add a route in **app.js** to retrieve the existing data from the database.

```
app.get('/editProduct/:id', (req,res) => {  
  const productId = req.params.id;  
  const sql = 'SELECT * FROM products WHERE productId = ?';  
  // Fetch data from MySQL based on the product ID  
  connection.query( sql , [productId], (error, results) => {  
    if (error) {  
      console.error('Database query error:', error.message);  
      return res.send('Error retrieving product by ID');  
    }  
    // Check if any product with the given ID was found  
    if (results.length > 0) {  
      // Render HTML page with the product data  
      res.render('editProduct', { product: results[0] });  
    } else {  
      // If no product with the given ID was found, render a 404 page or handle it accordingly  
      res.send('Product not found');  
    }  
  });  
});
```

Display Current Data in form

```
app.get('/editProduct/:id', (req,res) => {  
  const productId = req.params.id;  
  const sql = 'SELECT * FROM products WHERE productId = ?';  
  // Fetch data from MySQL based on the product ID  
  connection.query( sql , [productId], (error, results) => {  
    if (error) {  
      console.error('Database query error:', error.message);  
      return res.send('Error retrieving product by ID');  
    }  
    // Check if any product with the given ID was found  
    if (results.length > 0) {  
      // Render HTML page with the product data  
      res.render('editProduct', { product: results[0] });  
    } else {  
      // If no product with the given ID was found, render a 404 page or handle it accordingly  
      res.send('Product not found');  
    }  
  });  
});
```

app.js

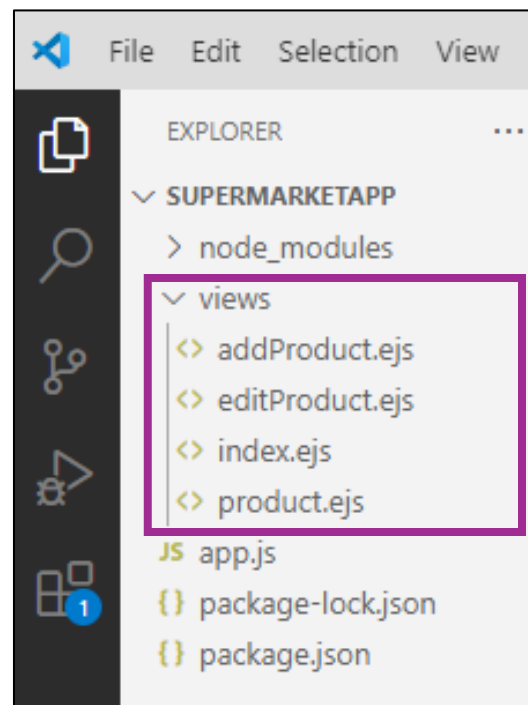
<!-- Edit Link -->

<td><a href="/editProduct/<%= products[i].productId %>">Edit</td>

index.ejs

Displaying product information

- Copy the '**editProduct.ejs**' file found in your **L17 Activity Files** folder and paste it into the **views folder** in your supermarket App.
- Your project folder should look like this:



Display Current Data in form

```
<% if (product) { %>
  <form action="/editProduct/<%= product.productId %>" method="POST">
    <label for="name">Product Name:</label><br>
    <input type="text" id="name" name="name" value = <%= product.productName %> required<br><br>

    <label for="quantity">Quantity:</label><br>
    <input type="number" id="quantity" name="quantity" min="0" step="1" value = <%= product.quantity %>
    required<br><br>

    <label for="price">Price:</label><br>
    <input type="number" id="price" name="price" min="0" step="0.01" value = <%= product.price %>
    required<br><br>

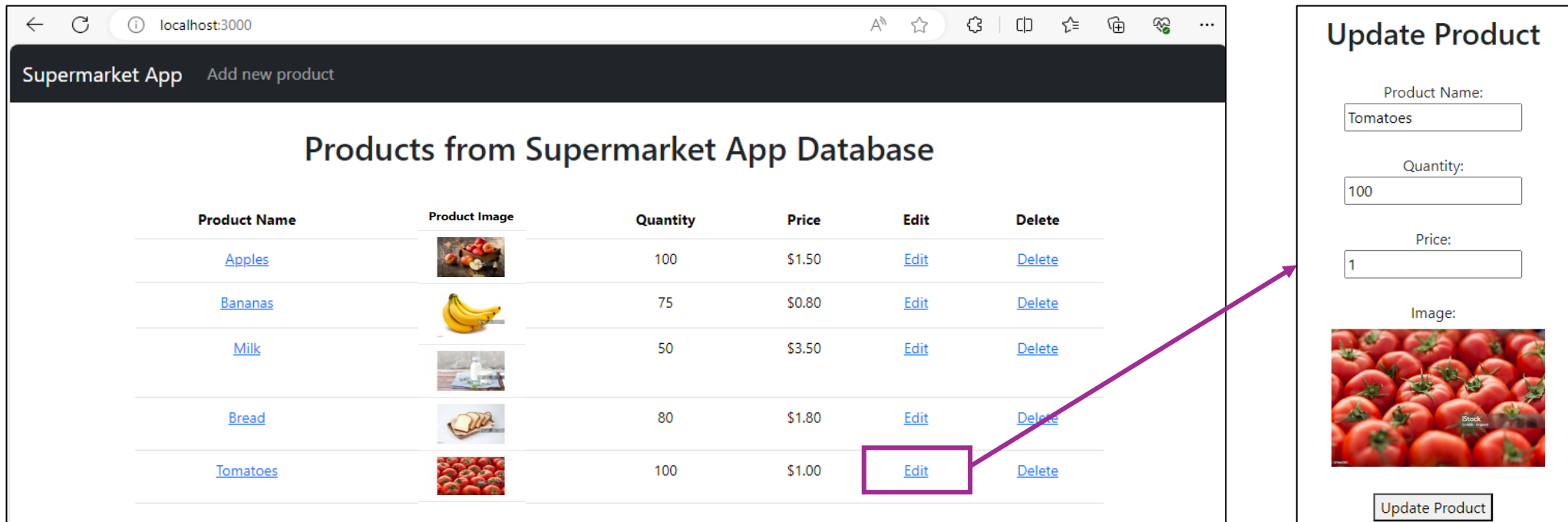
    <label for="image">Image:</label><br>
    <img src = '<%= product.image %>' width="20%"><br><br>

    <button type="submit">Update Product</button>
  </form>
<% } else { %>
```

editProduct.ejs

Let's test it out

- Save all your files.
- Open your terminal and enter “**npx nodemon app.js**” (if your server is not already running.)
- Open your browser and enter the url <http://localhost:3000>. You should see the following:



The screenshot displays the Supermarket App interface in a web browser. The browser's address bar shows `localhost:3000`. The app's header includes the title "Supermarket App" and a link "Add new product". The main content area is titled "Products from Supermarket App Database" and features a table of products. The table has columns for Product Name, Product Image, Quantity, Price, Edit, and Delete. The products listed are Apples, Bananas, Milk, Bread, and Tomatoes. The "Edit" link for Tomatoes is highlighted with a purple box, and a purple arrow points from this box to the "Update Product" form on the right. The form includes input fields for Product Name (filled with "Tomatoes"), Quantity (filled with "100"), and Price (filled with "1"), along with an image field showing a picture of tomatoes and an "Update Product" button.

Product Name	Product Image	Quantity	Price	Edit	Delete
Apples		100	\$1.50	Edit	Delete
Bananas		75	\$0.80	Edit	Delete
Milk		50	\$3.50	Edit	Delete
Bread		80	\$1.80	Edit	Delete
Tomatoes		100	\$1.00	Edit	Delete

Update Product

Product Name:

Quantity:

Price:

Image:


Updating Database

- Add a route in **app.js** to update the corresponding record in the database with the new data provided by the user.

```
app.post('/editProduct/:id', (req, res) => {  
  const productId = req.params.id;  
  // Extract product data from the request body  
  const { name, quantity, price } = req.body;  
  const sql = 'UPDATE products SET productName = ? , quantity = ?, price = ? WHERE productId = ?';  
  
  // Insert the new product into the database  
  connection.query( sql , [name, quantity, price, productId], (error, results) => {  
    if (error) {  
      // Handle any error that occurs during the database operation  
      console.error("Error updating product:", error);  
      res.send('Error updating product');  
    } else {  
      // Send a success response  
      res.redirect('/');  
    }  
  });  
});
```

*Study the code flow shown here.

Then, copy the complete implementation from the next slide into your application.

Updating Database

```
app.post('/editProduct/:id', (req, res) => {
  const productId = req.params.id;
  // Extract product data from the request body
  const { name, quantity, price } = req.body;
  const sql = 'UPDATE products SET productName = ? , quantity = ?, price = ? WHERE productId = ?';

  // Insert the new product into the database
  connection.query( sql , [name, quantity, price, productId], (error, results) => {
    if (error) {
      // Handle any error that occurs during the database operation
      console.error("Error updating product:", error);
      res.send('Error updating product');
    } else {
      // Send a success response
      res.redirect('/');
    }
  });
});
```

Let's test it out

- Save all your files.
- Open your terminal and enter “**npx nodemon app.js**” (if your server is not already running.)
- Open your browser and enter the url <http://localhost:3000>. Make some changes and click on “Update Product”:

Update Product

Product Name:

Quantity:

Price:

Image:


localhost:3000

Supermarket App Add new product

Products from Supermarket App Database

Product Name	Product Image	Quantity	Price	Edit	Delete
Apples		100	\$1.50	Edit	Delete
Bananas		75	\$0.80	Edit	Delete
Milk		50	\$3.50	Edit	Delete
Bread		80	\$1.80	Edit	Delete
Tomatoes		80	\$1.50	Edit	Delete



Delete a Product

Delete a product

- Add a route in **app.js** to delete a product from the database.

```
app.get('/deleteProduct/:id', (req, res) => {  
  const productId = req.params.id;  
  const sql = 'DELETE FROM products WHERE productId = ?';  
  connection.query( sql , [productId], (error, results) => {  
    if (error) {  
      // Handle any error that occurs during the database operation  
      console.error("Error deleting product:", error);  
      res.send('Error deleting product');  
    } else {  
      // Send a success response  
      res.redirect('/');  
    }  
  });  
});
```

Delete a product

```
app.get('/deleteProduct/:id', (req, res) => {  
  const productId = req.params.id;  
  const sql = 'DELETE FROM products WHERE productId = ?';  
  connection.query( sql , [productId], (error, results) => {  
    if (error) {  
      // Handle any error that occurs during the database operation  
      console.error("Error deleting product:", error);  
      res.send('Error deleting product');  
    } else {  
      // Send a success response  
      res.redirect('/');  
    }  
  });  
});
```

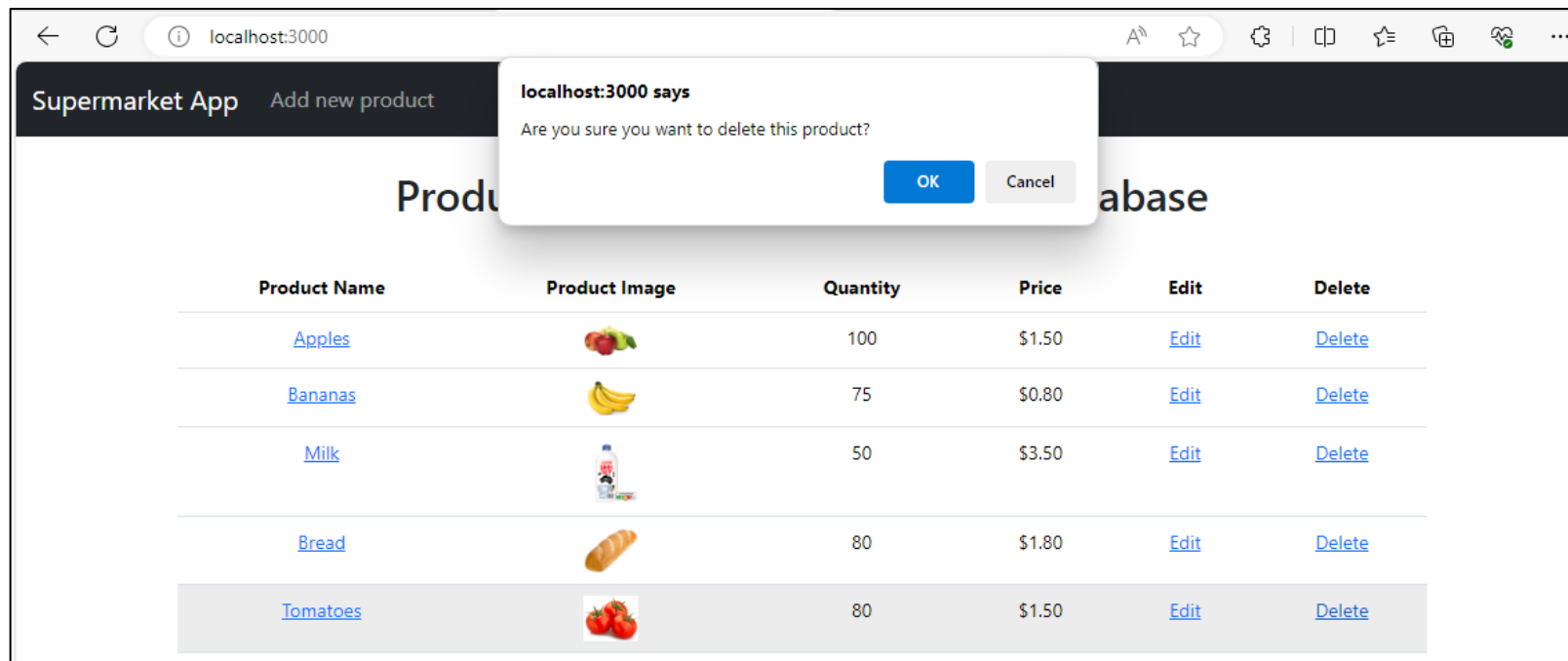
app.js

```
<!-- Delete Link -->  
<td><a href="/deleteProduct/<%= products[i].productId %>" onclick="return confirm('Are you sure you want to  
delete this product?')">Delete</a></td>
```

index.ejs

Let's test it out

- Save all your files.
- Open your terminal and enter “**npx nodemon app.js**” (if your server is not already running.)
- Open your browser and enter the url <http://localhost:3000> and try deleting one of the product. The product should be removed from the list after you select “ok”.

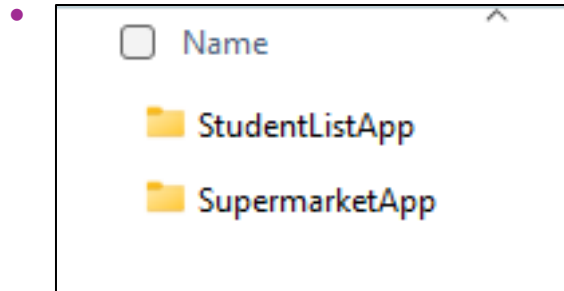


Activity

- **studentList App**
 - Include **Update** and **Delete** Function to your studentList App.
- Your **studentList App** should have all **CRUD function** by the end of today's lesson.

Deliverables

- Activity



- **Commit and Push** your code to **GitHub**
- Upload all deliverables in a zipped folder to POLITEMall **before the next lesson.**

What have you learnt?

- Database with Express and EJS
 - Update Data
 - Delete Data

End of Lesson 17